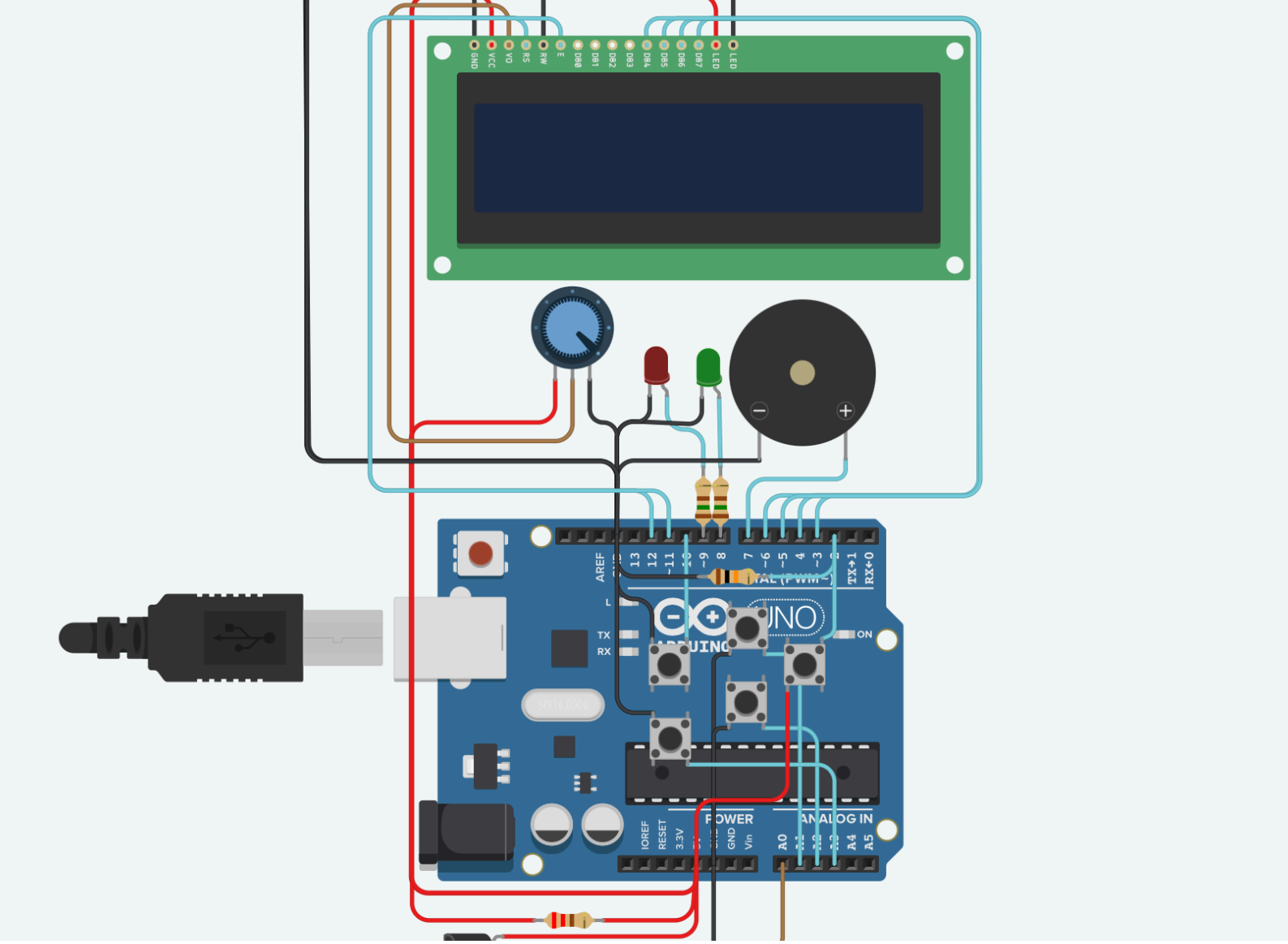


Microprocessors - COE 381

Smart Thermometer System

Group 24, Batch 3
27 March 2026

1. Gordon Yennutrin Laar - 7274623
2. Glah Patrick Edem - 7271523
3. Nii-Quaye Joseph Kotei - 7276823
4. Obeng Ransford Anane - 7278223
5. Paul Logodzo - 7275123
6. Michael Kwabena Konadu Okrah - 7279223
7. Oduro Godson Agyei - 7278623
8. Ithiel Nkansah - 7276923
9. Defiat Zion Selorm - 1821422

Batch 3, Group 24

2. Table of Contents

1.Group Information
2.Table of Contents
3.Problem Statement
4.Methodology
    4.1. System Approach
    4.2. Code Overview
    4.3. System Design & Schematics
5.Demonstration / Simulation
6.Conclusion

3. Problem Statement

Temperature monitoring is an essential aspect of healthcare, especially for detecting conditions such as fever or hypothermia. However, traditional thermometers have significant limitations. They only display temperature readings without offering intelligent features such as alerts, data storage, or trend analysis.

This creates a challenge in situations where continuous monitoring and quick decision-making are required. There is a need for a smarter system that not only measures temperature accurately but also analyzes data, alerts users of abnormal conditions, and stores historical records.

This project addresses these limitations by developing a **Smart Digital Thermometer System** that enhances basic temperature measurement with intelligent monitoring capabilities.

4. Methodology

4.1. System Approach

The system is designed as a **microcontroller-based embedded system** that integrates sensing, processing, and user interaction.

The approach includes:

- Collecting temperature data using an analog sensor.
- Converting analog signals into digital values using the microcontroller's ADC.
- Processing and validating temperature readings.
- Displaying results on an LCD screen.
- Triggering alerts when abnormal conditions are detected.
- Storing readings for analysis.
- Performing trend analysis after multiple readings.

A **state machine architecture** is used to manage different system modes such as:

- OFF Mode
- READ Mode
- MENU Mode
- SUBMENU Mode
- TREND Mode

This ensures efficient and organized system operation.

4.2. Code Overview

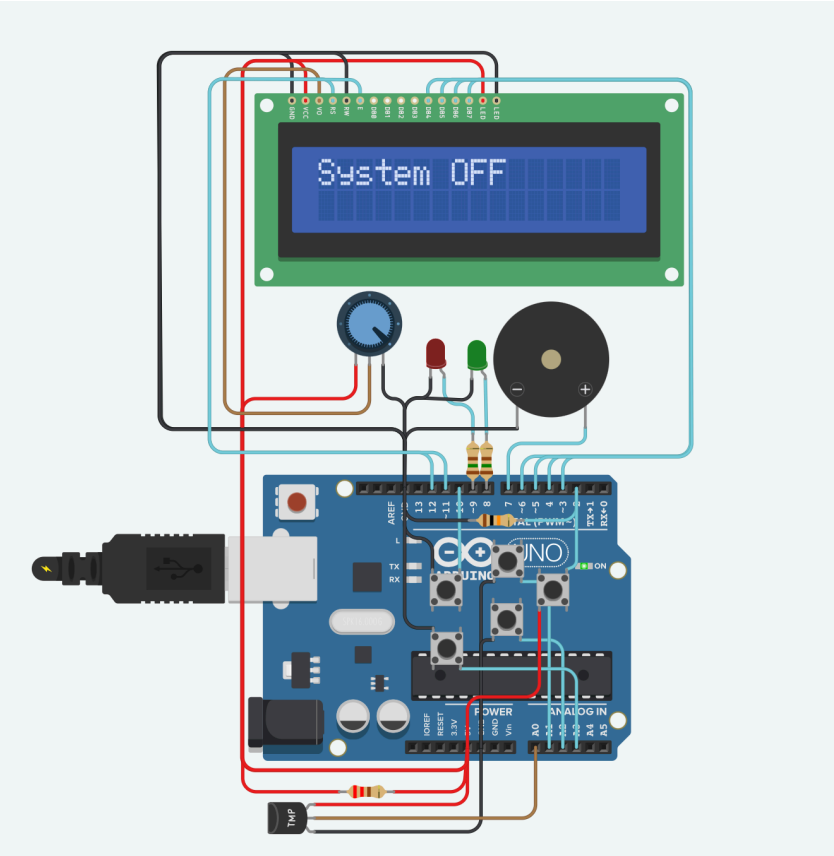
```
#include <LiquidCrystal.h>
#include <EEPROM.h>

// ================= LCD =================
LiquidCrystal lcd(12, 11, 5, 4, 3, 6);

// ================= PINS =================
#define TEMP_PIN A0
#define POWER_BTN 2
#define READ_BTN 10
#define BUZZER 7
#define RED_LED 9
#define GREEN_LED 8
#define MENU_UP_BTN A1
#define MENU_DOWN_BTN A2
#define BACK_BTN A3

// ================= SYSTEM STATE MACHINE =================
enum SystemState {
  STATE_OFF,
  STATE_READ,
  STATE_MENU,
  STATE_SUBMENU,
  STATE_TREND
};
```

The system logic is implemented using embedded Arduino environment. Key features of the code include:

- **Analog-to-Digital Conversion (ADC):**
  Reads temperature data from the LM35 sensor.
- **Data Averaging:**
  The system samples the sensor five times and computes the average to improve accuracy.

- **Validation Logic:**
  Ensures readings fall within the safe human range (30°C to 43°C). Invalid values are rejected.
- **Conditional Statements:**
  Used to compare readings with alert thresholds and trigger appropriate responses.
- **State Machine Implementation:**
  Controls system behavior across different modes.
- **Arrays:**
  Store multiple temperature readings for analysis.
- **Structures (Structs):**
  Organize stored data efficiently.
- **Timing Control (`millis()`):**
  Enables non-blocking delays for smoother operation.
- **Input Handling:**
  Button presses are detected through polling.

4.3. System Design and Schematics



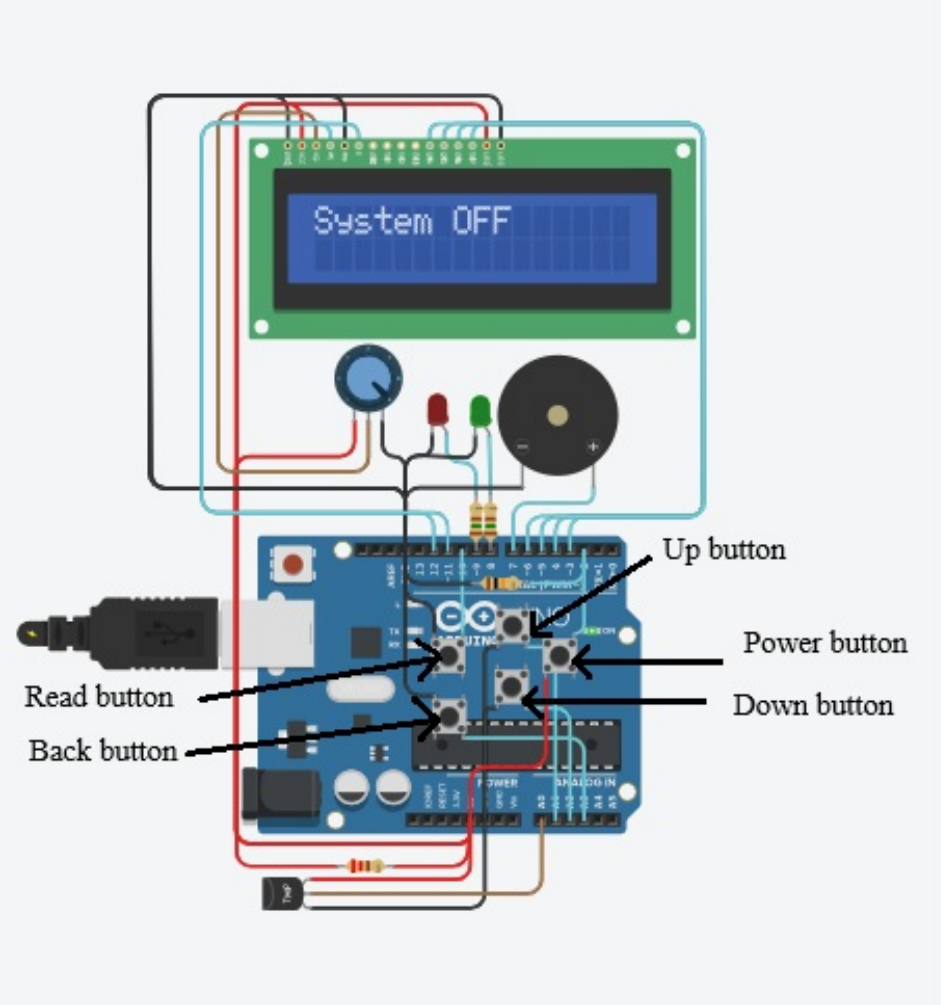
The system consists of the following hardware components:

- **Arduino Microcontroller -** Central processing unit.
- **LM35 Temperature Sensor** - Measures temperature
- **16x2 LCD Display** - Displays output and system messages.
- **Push Buttons** - For power and menu navigation.
- **Buzzer** - Provides audio alerts.
- **Red LED** - Indicates abnormal temperature.
- **Green LED** - Indicates normal temperature.

System Operation Flow:

1. Sensor measures temperature.
2. Arduino processes and validates the data.
3. LCD displays the result.
4. Alerts are triggered if thresholds are exceeded.
5. Data is stored for trend analysis.

5. Demonstration



This digital temperature monitoring system is designed to measure, display, store, and analyze human body temperature in a structured and user-friendly manner. The system remains in the OFF state until the power button is pressed, at which point it activates and enters reading mode. In this mode, the device waits for the user to press the read button to measure a patient's temperature. Once pressed, the sensor collects multiple samples to ensure accuracy, calculates the average temperature, and displays the result on the LCD screen. The system automatically checks whether the measured value falls within the safe temperature range. If the temperature is within the preset limits, the green LED lights up to indicate a normal condition. If the temperature exceeds the high limit or drops below the low limit, the red LED turns on and the buzzer sounds to alert the user.

Each valid reading is saved into the system's memory along with a timestamp, allowing up to ten previous records to be stored. When the memory becomes full, the oldest record is automatically replaced by the newest one. After five consecutive valid readings, the system performs a trend analysis to determine whether the patient's temperature is rising, falling, or stable. The result is displayed for a few seconds before the system automatically returns to reading mode.

The device also includes a menu system that allows the user to view stored temperature history, adjust the alert limits, switch between Celsius and Fahrenheit, or perform a factory reset. Navigation is done using the menu buttons, while the read button acts as a selection or confirmation key within the menu. Safety is built into the alert adjustment feature to ensure the low limit cannot exceed the high limit. Performing a factory reset restores all default settings and clears stored readings. Throughout operation, the LCD provides clear feedback, the LEDs indicate temperature status, and the buzzer provides audible alerts when necessary, ensuring the device operates as a reliable and interactive temperature monitoring system.

5.1. Simulation Process

During the demonstration:

- The system is powered ON using the power button.

- The LCD initializes and displays **Read Mode**.
- The user presses the Read button to capture temperature.

5.2. System Outputs

- **Normal Temperature:**
    - Displayed on LCD.
    - Green LED turns ON.
    - No sound alert.
- **Abnormal Temperature:**
    - Displayed on LCD.
    - Red LED turns ON.
    - Buzzer activates.
- **Invalid Temperature:**
    - LCD displays “INVALID”.

- **Trend Analysis:**
  After five readings, the system indicates whether the temperature is:
    - Rising
    - Falling
    - Stable

5.3. Key Takeaways

- The system provides more than just readings; it offers intelligent feedback.
- Alerts improve response time to abnormal conditions.
- Data storage enables monitoring over time.
- User interaction enhances usability and flexibility.

The Smart Digital Thermometer System successfully addresses the limitations of traditional thermometers by integrating intelligent monitoring features.

6. Conclusion

Findings

- Accurate temperature measurement was achieved using the LM35 sensor.
- The system effectively detects abnormal conditions and triggers alerts.
- Trend analysis provides valuable insights into temperature changes.

Challenges Encountered

- Managing multiple system states efficiently.
- Ensuring user-friendly menu navigation.

Possible Improvements

- Integration of a real-time clock for timestamped data.
- Addition of Bluetooth or Wi-Fi connectivity.
- SD card support for long-term data storage.
- Development of a mobile application interface.